

Primers and Shortcuts: Testing Structural Preambles for LLM Graph Reasoning at Scale

Gal Barak

github.com/GalBrk

Nitzan Zacharia

github.com/NitzanZacharia

Inbal Moryles

github.com/iinbal

Abstract

Prepending a short *primer* of factual sentences about each node’s local structure — degree, clustering coefficient, a random-walk structural encoding, or connected-component membership — before a graph’s standard text encoding is an intuitively appealing way to ease graph-reasoning prompts for a large language model. Any apparent gain is illusory, however, if a solver can recover the answer directly from the primer text without ever consulting the graph, and it is confounded if the primer merely lengthens the prompt. We introduce a graph-blind shortcut solver as a content-only lower bound, a length-matched filler control as a length-only upper bound, and a screening procedure — a difficulty ladder, a graph-free reading-limit probe, and a degree-preserving rewiring test — that determines, for each model, where a primer effect is even interpretable before one is measured. Evaluated at $n = 40$ on qwen3-1.7b and qwen3-4b, with and without chain-of-thought: a length-matched filler alone costs the thinking arm 11.7 points on dense graphs, and exactly one (primer, task) cell withstands every control — clustering on node_degree, a gain of +3.8 percentage points, replicated at +4.3 points on an independent sample ($p = 0.0006$).

1 Introduction

Fatemi et al. (2023) demonstrate that a large language model’s accuracy on graph-reasoning questions depends substantially on how the graph is encoded as text, independent of the model’s underlying reasoning ability. Their study varies the encoding function itself — adjacency lists, incident lists, and a family of “themed” encodings that rename nodes as people, places, or characters — while holding the question and the underlying graph fixed.

This work poses a complementary question: rather than altering how edges are described, does

prepending a short *primer* of derived, node-level facts before the graph encoding help a model answer questions about that graph? A primer lies outside the encoding vocabulary studied by Fatemi et al. (2023): it is additional, redundant information computed from the graph and expressed in plain sentences, a term this project introduces for it. If a primer condition helps only because it lets the model read the answer directly from the primer text rather than reason over the graph, that is not evidence of increased graph reasoning; it is evidence that the primer leaked the answer. And if a primer helps or hurts only because it lengthens or shortens the prompt, that is not evidence about its *content* either: Liu et al. (2024) and the needle-in-a-haystack protocol of Kamradt (2023) show that context position and length alone move retrieval accuracy, independent of what the added text says.

This paper makes three contributions. First, a single, deterministic primer renderer that produces seven conditions (Section 3.1) differing only in content, so that differences between conditions are interpretable as content effects rather than formatting artifacts. Second, a *shortcut solver* (Section 3.2) that answers each GraphQA task from rendered primer text alone, with no access to the graph, yielding a principled lower bound that any genuine primer benefit must exceed. Third, a screening design (Section 3.4) that determines, for a given model, where a primer effect is even interpretable in the first place: a graph-length/answer-magnitude ladder, a graph-free reading-limit probe, and a degree-preserving rewiring procedure that isolates a primer’s content effect from prompt length by construction. We evaluate all three within a density sweep of GraphQA tasks on two model sizes, qwen3-1.7b and qwen3-4b (Section 4).

2 Related Work

Fatemi et al. (2023) introduce GraphQA, a bench-

mark of graph reasoning tasks (node degree, node count, edge count, edge existence, connected nodes, cycle check, and disconnected nodes) paired with a family of graph-to-text encoding functions, and show that encoding choice, task, and the underlying graph generator all materially affect LLM accuracy. This project adopts their task definitions and their finding that the graph generator matters as motivation for fixing graph size and edge density explicitly in our own sweep design (Section 4), rather than sampling graph structure incidentally. We verified this directly before finalizing the design: sweeping the generator itself (Erdős–Rényi (Erdős and Rényi, 1959), Barabási–Albert, random regular, Watts–Strogatz, and stochastic block models) at matched node and edge counts moved the majority-class baseline from 0.15 to 1.0, a swing six times larger than any primer effect measured in this project, with random-regular graphs degenerate in the sense that every instance shares a single gold answer. A generator-family sweep was accordingly withdrawn: it confounds structural family with task difficulty rather than isolating a structural effect.

A broader literature probes LLM graph reasoning more generally. Wang et al. (2023) evaluate eight graph tasks of increasing complexity and report that LLMs are brittle to spurious correlations in graph and problem structure, the same concern that motivates our shortcut solver (Section 3.2), applied here to the primer channel rather than to the graph encoding. Guo et al. (2023) instead encode graphs through standard graph description languages (GML, GraphML) and find that LLMs struggle to exploit that structure directly, and Jin et al. (2024) survey the broader space of LLM-graph integration methods within which this project sits. Dziri et al. (2023) argue that transformers solve compositional tasks — multi-digit multiplication, dynamic programming — by linearized sub-graph matching rather than systematic multi-step reasoning, a finding that raises the same question a primer’s apparent benefit raises: is an observed gain evidence of reasoning, or evidence of a shorter pattern to match against? Geirhos et al. (2020) frame this concern as *shortcut learning* in general: a decision rule that performs well on the benchmark it was measured against without solving the intended problem. Our shortcut solver operationalizes precisely this concern for the primer channel, rather than for the model as a whole.

The RWSE primer condition (Section 3.1) adapts

the random-walk structural encoding of Dwivedi et al. (2022), originally developed for graph transformers rather than for text prompts; we render its return probabilities as natural-language sentences rather than feeding them to an attention mechanism. The chain-of-thought variant of our sweep follows Wei et al. (2022), who show that eliciting step-by-step reasoning changes what a model can accomplish on structured tasks, consistent with the reasoning-mode effect we observe in Section 5, which dwarfs every primer effect measured. Our significance-testing methodology (Section 3.5) follows the general recommendation of Dror et al. (2018) to test the actual dependence structure of paired evaluation data rather than treat each item as independent.

3 Method

3.1 Primers

A primer is a short preamble of factual sentences that details a graph’s local or global structure, prepended to the standard text encoding and the task question. To ensure that observed differences reflect genuine content effects rather than formatting artifacts, a single renderer produces every condition: all node-level facts consequently share identical sentence framing (e.g., “Node N has...”), and floating-point values are rounded and formatted identically across conditions. We evaluate seven distinct conditions:

- **None:** The baseline graph text encoding without a primer.
- **Components:** A single graph-level sentence stating the number of connected components.
- **Degree:** The exact degree reported for each node.
- **Clustering:** The clustering coefficient computed for each node.
- **RWSE:** The random-walk structural encoding (Dwivedi et al., 2022) return probabilities for each node.
- **All:** The degree, clustering, and RWSE statistics combined for every node.
- **Filler:** A content-free control matched in length to the factual primers (e.g., “Node N is simply present within the graph G.”), isolating the effect of primer content from the confound of merely lengthening the prompt.

3.2 Shortcut solvers

For a primer condition’s measured accuracy on a task to constitute evidence of graph reasoning, it must exceed what is recoverable from the primer text alone. We construct a shortcut solver that takes only rendered primer text as input — never the graph object, and never the full-precision statistics the primer rounds away — and answers each GraphQA task directly. It combines sixteen exact rules (for instance, deriving edge count algebraically from a `degree` primer), one heuristic rule, eight fitted rules whose fitting and scoring graph sets are kept structurally disjoint so that accuracy is not inflated by evaluating on the fitting data, and an exact enumeration bound for small graphs. Because the solver is structurally incapable of observing the graph, its accuracy on a given (primer condition, task) cell constitutes a model-independent bound: any model result on that cell should be interpreted relative to this *shortcut ceiling* rather than to zero, since a primer condition that fully determines the answer (shortcut accuracy of 1.0) renders model accuracy on that cell uninformative about graph reasoning, however high it may be.

3.3 Prompt assembly

Each prompt concatenates the primer, the graph’s text encoding, and the task question, followed by a chain-of-thought suffix when the model variant requires one. Prompts are rebuilt from a canonicalized, parsed representation of each graph rather than reused verbatim from the source dataset, so that node and edge ordering is reproducible and independent of the order in which the original question happened to list them.

3.4 Locating an interpretable test: ladder, retrieval probe, and rewiring

A content effect and a length effect are not the only quantities a primer delta can measure. A model that cannot read the prompt at all is neither helped nor harmed by a primer’s content, and a model that reads perfectly but is not actually taxed by the task offers a primer no room to show an effect. Both failure modes yield a null result indistinguishable from “the primer has no effect” unless they are ruled out beforehand. We rule them out with three linked tools rather than one, since they fail in opposite directions.

The ladder. We screen every model across 18 *rungs*, cells of an $(n, \bar{k})$ grid spanning graph length $n \in \{20, \dots, 400\}$ nodes and mean degree (answer magnitude) $\bar{k} \in \{4, \dots, 24\}$, on the `node_degree` task under `none` alone. These two axes induce difficulty with opposite implications for a primer: a *length-limited* rung means the model cannot read the graph and therefore cannot exploit a primer either, whereas a *magnitude-limited* rung means the model reads the graph without difficulty yet still miscounts, precisely the failure a primer could circumvent. A one-dimensional size sweep can expose only the former. A rung is discarded prior to any generation if it is *blind* (a majority-class guesser scores above 0.25), *silent* (the primer’s rendered value has a standard deviation below 0.10, rendering it effectively constant and uninformative at any sample size), or *unreadable* (beyond the model’s measured reading limit, defined below). A rung that survives all three gates is *valid*, the only kind for which a primer test is meaningful.

Retrieval probe. The reading limit against which each rung is screened comes from a separate, graph-free needle-in-haystack probe (Kamradt, 2023; Liu et al., 2024): a target statement is embedded among $k \in \{160, \dots, 640\}$ distractor statements at varying depth, with no graph present in the prompt, isolating raw token-position retrieval from graph reasoning entirely.

Rewiring. Even once a valid rung is identified, the primer test itself must still avoid the length confound described above, since adding primer content necessarily adds characters. We resolve this with a degree-preserving double edge swap (Fosdick et al., 2018): removing edges (a, b) and (c, d) and adding (a, d) and (c, b) preserves every node’s degree exactly, leaving the degree sequence, every gold answer, and the majority-class baseline unchanged; and because the `incident` encoder emits one line per node, the rendered prompt retains the *same string length* before and after the swap. What changes is triangle count and clustering: measured at $n=80$, $m=334$, transitivity rises from 0.000 to 0.630. This yields a within-instance paired design (the same graph, rewired, with the same question and the same gold answer) that isolates a primer’s content effect from length by construction, rather than through the post-hoc control the `filler` condition (Section 3.1) provides when rewiring is not used.

3.5 Significance testing

Following the general caution of [Dror et al. \(2018\)](#) against treating dependent evaluation items as independent, every comparison reported here is a paired test clustered by *graph* rather than by (task, index): two prompts constructed from the same underlying graph are not independent draws, and treating them as such understates variance and overstates significance. We employ a clustered paired permutation test (sign-flipping whole graph clusters under the null, computed exactly below 20 clusters and via Monte Carlo with add-one correction otherwise) together with a clustered paired bootstrap for confidence intervals on the effect size, and correct across the resulting family of comparisons with the Benjamini–Hochberg procedure ([Benjamini and Hochberg, 1995](#)). Individual (density, task, condition) cells are additionally reported against `none` with an exact McNemar test ([McNemar, 1947](#)), which is well powered at the per-cell sample sizes used here but not across the pooled family, a role the clustered test is designed to fill.

4 Experimental Setup

Tasks. We evaluate on six GraphQA tasks from [Fatemi et al. \(2023\)](#): node count, edge count, node degree, connected nodes, edge existence, and cycle check.

Models. We compare `qwen3-1.7b` and `qwen3-4b` from the Qwen3 model family ([Qwen Team, 2025](#)), each evaluated in a non-thinking (greedy decoding, no chain-of-thought) and a thinking (chain-of-thought-enabled; [Wei et al., 2022](#)) variant, for four model arms in total.

Sweep design. All four arms are evaluated on one shared set of prompts, so that every model receives byte-identical inputs. Graphs are Erdős–Rényi ([Erdős and Rényi, 1959](#)) with a fixed node count of 40 and edge density swept over four levels ($p \in \{0.10, 0.20, 0.35, 0.50\}$). Every combination of the seven primer conditions (Section 3.1) and six tasks is evaluated on 100 graphs per (density, task) cell, yielding 16,800 prompts per model arm. Generation is greedy, with a raised output budget (up to 8192 tokens), since at the densest level tasks such as edge count require enumerating several hundred edges in the answer.

Metrics. We use exact match for integer- and boolean-valued tasks and set-F1 for connected

nodes. Every result is reported against both the `none` baseline (Section 3.5) and the shortcut ceiling for the corresponding (primer condition, task) cell (Section 3.2). Generations that reach the output-token cap are excluded from scoring rather than scored as incorrect, and are reported separately as a per-cell drop count.

Known validity caveats. At this graph size and density range, several cells are known in advance to be shortcut-dominated rather than genuine tests of graph reasoning: any primer with one sentence per node allows node count to be recovered by counting sentences; cycle check’s gold answer is “yes” for nearly every graph past the sparsest density level; and the `degree` primer states the node-degree answer verbatim. The one cell already established as free of this contamination at this graph size is node degree under `{none, components, clustering, filler}`. We report and interpret results with this caveat in mind rather than omitting the contaminated cells, since a null or negative result on an uncontaminated cell remains informative in its own right.

Node naming. In addition to the default integer node ids, we substitute a fixed set of 40 Game-of-Thrones character names¹ (`graphtalk/node_naming.py`) for the integer node ids GraphQA otherwise uses, assigned by node index rather than by any property of the graph. Substitution and desubstitution are implemented as a text pass around the existing, unmodified prompt-building and scoring code: an id is replaced only where it follows the literal word “node”/“Node” (never a count, a coefficient, or a rendered statistic), and a model’s response is converted back to integers before it reaches the same scorer used for the integer-id runs. This tests whether the primer/shortcut relationship reported here depends on node-label surface form, the same

¹By node index: 0 Ned, 1 Catelyn, 2 Daenerys, 3 Jon, 4 Bran, 5 Sansa, 6 Arya, 7 Cersei, 8 Jaime, 9 Petyr, 10 Robert, 11 Jorah, 12 Viserys, 13 Joffrey, 14 Maester, 15 Theon, 16 Rodrik, 17 Lysa, 18 Stannis, 19 Osha, 20 Tyrion, 21 Brienne, 22 Davos, 23 Varys, 24 Tormund, 25 Podrick, 26 Missandei, 27 Margaery, 28 Loras, 29 Oberyn, 30 Ellaria, 31 Melisandre, 32 Gendry, 33 Meera, 34 Jojen, 35 Ygritte, 36 Grenn, 37 Samwell, 38 Gilly, 39 Shireen. The first 20 names are [Fatemi et al. \(2023\)](#)’s own vendored list (character 1 respelled “Catelyn” in full, from the vendored abbreviation “Cat”, which collides with an ordinary English word); the remaining 20 are added by this project (`graphtalk/node_naming.py`) to cover every node at $n=40$, since the vendored list extends only to 20 names – sufficient for the original benchmark, which never requires more.

Sample	Δ acc.	n (disc.)	p
Original	+3.8pp	369	0.0017
Replication	+4.3pp	—	0.0006

Table 1: qwen3-1.7b, task `node_degree`, primer `clustering` vs. `none`, $n=40$, pooled over densities $p \leq 0.50$. The only (primer, task) cell in this project that withstands the shortcut ceiling (bar 0.08, unrelated to the effect), the length-matched `filler` control, and independent replication on a fresh graph sample. The replication’s discard count was not tracked separately from the original sample, hence “—”.

axis [Fatemi et al. \(2023\)](#) vary with their themed encodings. As with the main density sweep, this condition has not yet been run through the model pipeline; results are reserved in Section 5.2.

5 Results

5.1 The one cell that survives every control

Table 1 reports the single (primer, task) cell that clears every control applied so far: the `clustering` primer yields a measurable improvement for qwen3-1.7b on `node_degree`, and the effect reproduces on an independent sample of graphs. This is a modest effect relative to the size of the comparison family (Section 3.5), and it constitutes a single cell out of the $6 \times 6 \times 4 \times 4$ family the full sweep covers below; it is reported here as the one result already confirmed, not as representative of the sweep as a whole.

5.2 Main sweep: qwen3-1.7b / qwen3-4b density sweep and GoT naming

6 Discussion

Two effects already established, from the same qwen3-1.7b / qwen3-1.7b-think pair used for the headline result (not a different, larger model), set the scale against which every primer effect in Table 2 must eventually be read. First, a length-matched `filler` primer — no structure, no shortcut, merely 1,831 characters — costs the thinking arm 6.3 points pooled and 11.7 points on dense graphs, exceeding most primer effects measured anywhere in this project. Second, enabling chain-of-thought on the identical checkpoint and identical prompts yields a gain of +2.9 to +50.0 points as density rises (+29.2 pooled over 9,439 pairs), an order of magnitude larger than any primer effect on record, including the headline result in Table 1. Any effect attributed to a primer’s con-

tent, particularly in the thinking arms, should be read against both of these before being interpreted as evidence of graph reasoning; this is also why the rewiring design of Section 3.4 exists, to enforce that comparison by construction rather than through post-hoc control.

Limitations

This sweep covers only two model sizes (1.7B and 4B parameters) from a single model family, so the findings may not generalize to substantially larger models or to other architectures. Several (primer condition, task) cells are known in advance to be shortcut-dominated at this graph size and density range (Section 4); results on those cells bound what can be concluded about genuine graph reasoning even though we report them. Graphs are drawn from a single generator family (Erdős–Rényi); whether primer effects transfer to other graph generators, which [Fatemi et al. \(2023\)](#) show matters for encoding effects, remains untested here, and, as noted in Section 2, we found directly that varying the generator family itself is not a safe way to test this, since doing so confounds structure with task difficulty. Compute and time constraints limited this sweep to a single node-count/density design rather than a broader sweep over graph sizes; the ladder of Section 3.4 locates where a broader design would be interpretable, but has not itself yet been run as a primer test.

The ladder, retrieval probe, and rewiring mechanism (Section 3.4) are implemented but have not yet been run to completion on qwen3-1.7b/qwen3-4b themselves: no rung has yet been confirmed valid for either model, so the primer-*content* rewiring test has not yet been exercised on any comparison. Both are follow-up work rather than a reported result here.

This project’s node-naming substitution (Game-of-Thrones character names in place of integer ids) is implemented and tested, and is included above as a reserved result (Section 5.2); whether it alters the primer/shortcut relationship reported here, the same axis [Fatemi et al. \(2023\)](#) vary with their themed encodings, remains unknown at the time of writing.

References

Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the*

Model	Task	none	comp.	degree	clust.	rwse	all	filler
[TODO: qwen3-1.7b / qwen3-1.7b-think / qwen3-4b / qwen3-4b-think $\times$ 6 tasks $\times$ 4 densities ($p = 0.10, 0.20, 0.35, 0.50$), accuracy against none (McNemar) and against the shortcut ceiling (Section 3.2)]								

Table 2: Reserved for the full $n=40$ density-sweep results across all four model arms, six tasks, seven primer conditions, and four density levels.

- Royal Statistical Society: Series B (Methodological)*, 57(1):289–300.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1383–1392.
- Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2022. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations (ICLR)*.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. 2023. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Paul Erdős and Alfréd Rényi. 1959. On random graphs i. *Publicationes Mathematicae Debrecen*, 6:290–297.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*.
- Bailey K. Fosdick, Daniel B. Larremore, Joel Nishimura, and Johan Ugander. 2018. Configuring random graph models with fixed degree sequences. *SIAM Review*, 60(2):315–355.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673.
- Jiayan Guo, Lun Du, and Hengyu Liu. 2023. Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking. *arXiv preprint arXiv:2305.15066*.
- Bowen Jin, Gang Liu, Chi Han, Meng Jiang, Heng Ji, and Jiawei Han. 2024. Large language models on graphs: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(12):8622–8642.
- Greg Kamradt. 2023. Needle in a haystack – pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paran-jape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.
- Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.